Zhi-Hua Zhou

Machine Learning

Translated by Shaowu Liu

The slides of this chapter are translated by Yi-Fan Ma

Springer

Chapter 11
Feature Selection and Sparse Learning

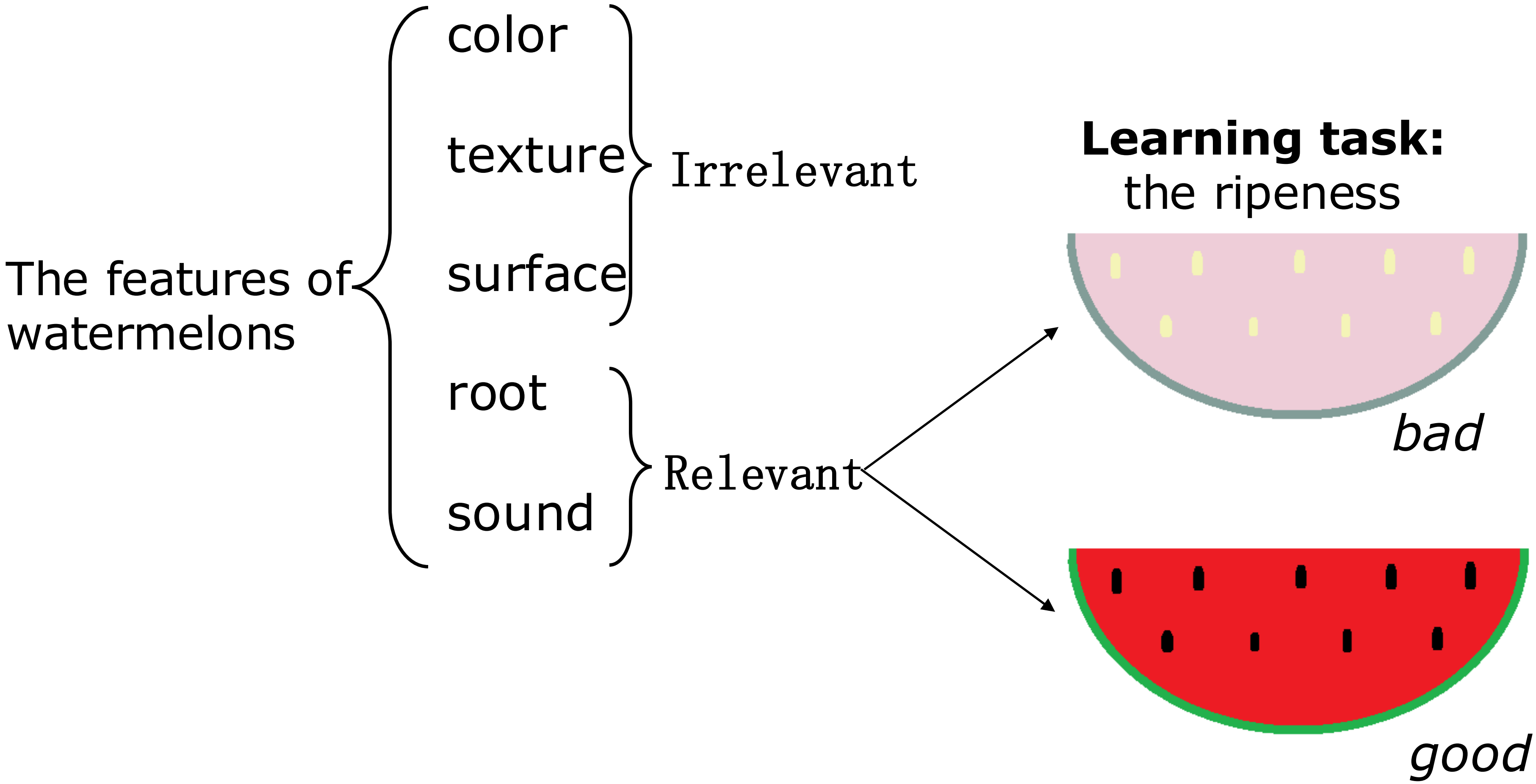
Feature

- Feature
  - In machine learning, attributes are called *features*.

- Different types of features
  - Relevant features: useful ones for the learning task
  - Irrelevant features: useless ones for the learning task
  - Redundancy features*: the information they contains can be deduced from other features

*For simplicity, redundancy features will not be discussed in this chapter

An example: the features of watermelons



Feature selection

- Feature selection
  - Select irrelevant features from the given feature set.
  - Make sure that important features will not be discarded.

- Reasons
  - Alleviate the curse of dimensionality: building the model based on fewer attributes.
  - Reduce the difficulty of learning: keeping the key information.

An example: feature selection in the task of determining the ripeness

The features of watermelons

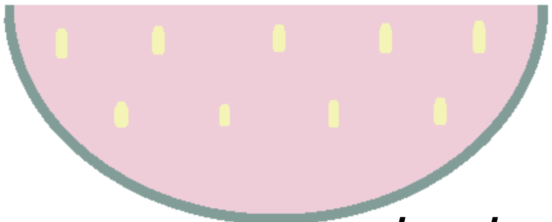
color
texture
surface
root
sound

Irrelevant

Relevant

Feature selection: selecting relevant features for the task

Learning task:
the ripeness



bad

good

General method for feature selection

- Evaluate all possible feature subsets
  - Impractical since suffering from the combinatorial explosion

- A more practical method

Two key steps: *subset search* & *subset evaluation*

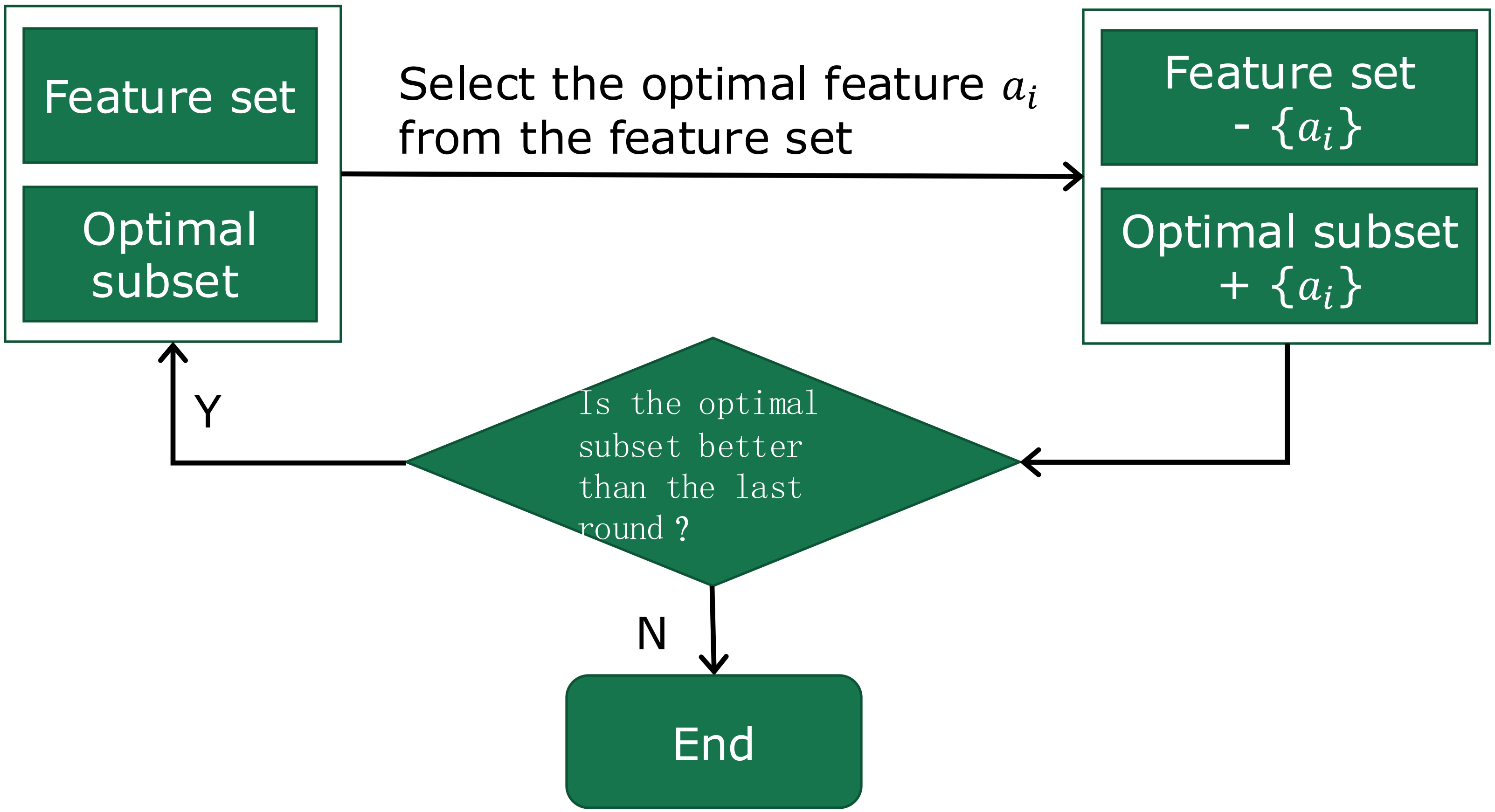
Subset search

Select the feature subset that contains important information with greedy search.

- Forward search: add relevant features
- Backward search: remove irrelevant features
- Bidirectional search: add relevant features and remove irrelevant features simultaneously

Forward search

- The optimal subset is initialized with an empty set, and the feature set is initialized with all the given features.



Subset evaluation

- Each feature subset corresponds to a partition of the data set
  - Each partition corresponds to a value assignment of the feature subset

- The label gives the ground truth partition

We can evaluate the feature subset by estimating the difference between these two partitions. The smaller the difference, the better the feature subset.

Evaluate the subset with information entropy

- The feature subset A determines a partition of the data set D
  - The data set D is split into V subsets according to those feature values on A, and each subset is denoted as D^v.
  - $\mathrm{Ent}(D^v)$ denotes the information entropy on D^v.
- The label Y gives the ground truth partition of data set D
  - $\mathrm{Ent}(D)$ denotes the information entropy D

The definition of the information entropy on D is:

$$\mathrm{Ent}(D) = -\sum_{k=1}^{|\mathcal{Y}|} p_k \log_2 p_k$$

p_i denotes the portion of the ith class

The information gain of feature subset A is:

$$\mathrm{Gain}(A) = \mathrm{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|}\mathrm{Ent}(D^v)$$

Commonly used feature selection methods

The feature selection method can be obtained by combining the feature subset search method and the subset evaluation method.

There are three kinds of commonly used feature selection methods:

- Filter methods
- Wrapper methods
- Embedding methods

Filter methods

Filter methods select features without considering the consequent learners.

- Relief (Relevant Features) [Kira and Rendell, 1992]
  - Incorporate a *relevance statistic* to measure the feature importance.
  - The importance of a feature subset is determined by the sum of the corresponding relevance statistic components.
  - Select features with a component greater than a user-specified threshold.
  - Alternatively, select features corresponding to the k largest components.

How to determine the relevance statistic?

Determine the relevance statistic in Relief

- Near-hit: $\boldsymbol{x}_{i,nh}$ is the nearest neighbor of $\boldsymbol{x}_i$ that has the same label
- Near-miss: $\boldsymbol{x}_{i,nm}$ is the nearest neighbor of $\boldsymbol{x}_i$ that has a different label
- The relevance statistic component of feature j is:

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i,nh}^j)^2 + \text{diff}(x_i^j, x_{i,nm}^j)^2$$

When j is discrete, $\text{diff}(x_a^j, x_b^j) = 0$ if $x_a^j = x_b^j$ and 1 otherwise;
When j is continuous, $\text{diff}(x_a^j, x_b^j) = |x_a^j - x_b^j|$, where x_a^j, x_b^j are normalized to $[0,1]$.

- Bigger relevance statistic component means that $\boldsymbol{x}_{i,nh}$ is nearer than $\boldsymbol{x}_{i,nm}$ on feature j. Thus, feature j is more useful to distinguish different classes.
- The computational complexity of Relief is linear to the number of sampling and the number of original features, and hence it is an efficient method.

Extend Relief to multi-class

Relief is designed for binary classification, and its variant Relief-F[Kononenko, 1994] can handle multi-class classification

- Given a data set with $|\mathcal{Y}|$ classes. For a sample $\boldsymbol{x}_i$ with label k
- Near-hit: $\boldsymbol{x}_{i,nh}$ is the nearest neighbor of $\boldsymbol{x}_i$ that has the same label k
- Near-miss: find a near-miss from each class other than k, denoted by $\boldsymbol{x}_{i,l,nm}(l = 1,2,\ldots,|\mathcal{Y}|; l \neq k)$
- The relevance statistic component of feature k is:

$$\delta^j = \sum_i -\mathrm{diff}(x_i^j, x_{i,nh}^j)^2 + \sum_{l \neq k} \left(p_l \times \mathrm{diff}(x_i^j, x_{i,l,nm}^j)^2 \right)$$

p_l is the portion of class l samples in the data set D

Wrapper methods

Wrapper methods directly use the performance of subsequent learners as the evaluation metric for the feature subset

- Wrapper methods aim to find the most useful feature subset “tailored” for the given learner.

- Wrapper methods are usually better than filter methods in terms of the learner’s final performance since the feature selection is optimized for the given learner.

- Wrapper methods are often much more computationally expensive since they train the learner multiple times during the feature selection.

LVW method

LVW(Las Vegas Wrapper)[Liu and Setiono, 1996] searches feature subsets using a **randomized strategy** under the framework of the Las Vegas method, and the subsets are evaluated based on the final classification error.

- Generate random feature subset

- Estimate the error of learner using cross-validation on the feature subset

- Perform multiple times, and select the feature subset with the smallest error from multiple randomly generated feature subsets as the final solution*.

*LVW may return nothing if the running time is limited.

Embedding methods

Embedded methods unify the feature selection process and the learner training process into a joint optimization process, that is, the features are automatically selected during the training.

Embedding methods

Embedded methods unify the feature selection process and the learner training process into a joint optimization process, that is, the features are automatically selected during the training.

- For a linear regression model with squared error, and L_2 regularization is used to prevent overfitting. The optimization objective is :

$$\min_{\boldsymbol{w}} \sum_{i=1}^{m} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 + \lambda \|\boldsymbol{w}\|_2^2$$

Ridge regression [Tikhonov and Arsenin, 1977]

Embedding methods

Embedded methods unify the feature selection process and the learner training process into a joint optimization process, that is, the features are automatically selected during the training.

- For a linear regression model with squared error, and L_2 regularization is used to prevent overfitting. The optimization objective is :

$$\min_{\boldsymbol{w}} \sum_{i=1}^{m} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 + \lambda \|\boldsymbol{w}\|_2^2$$

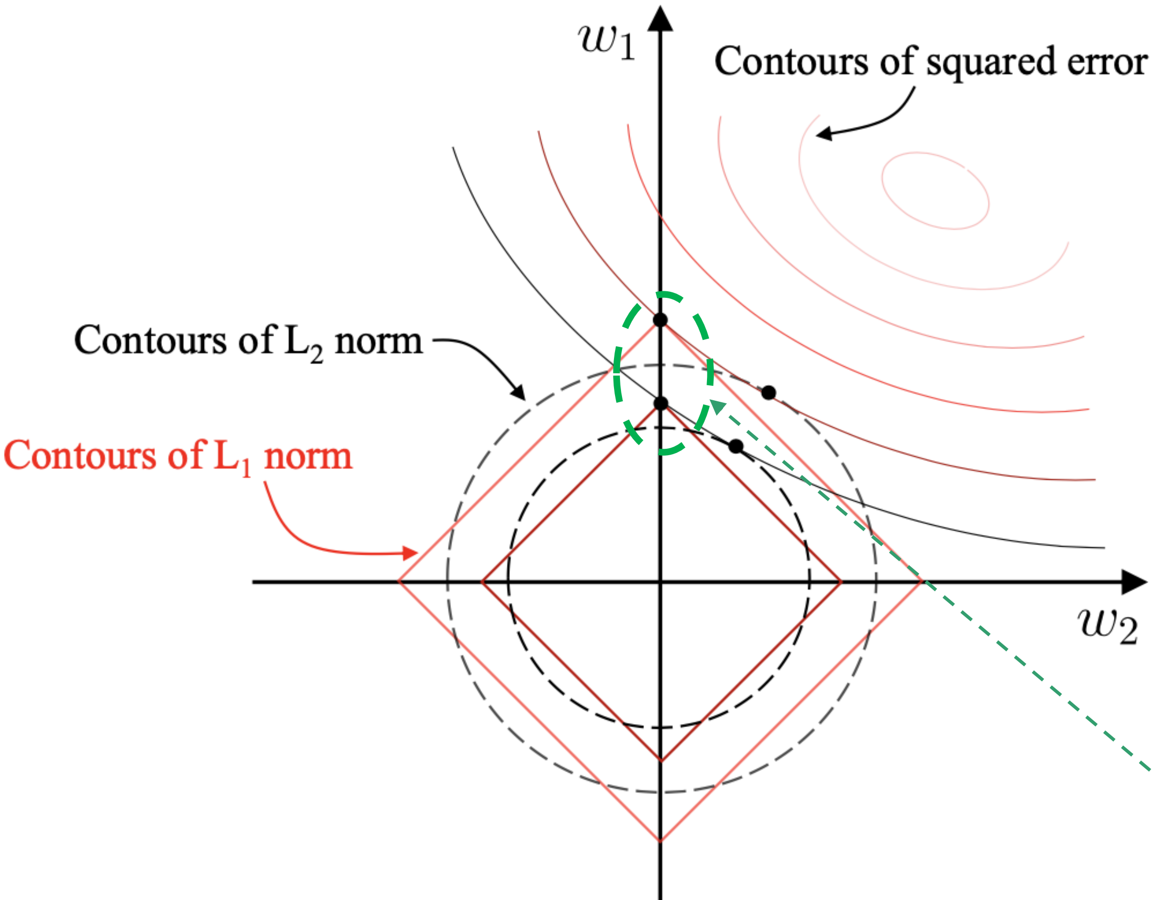
Ridge regression [Tikhonov and Arsenin, 1977]

- **LASSO** [Tibshirani, 1996] can be obtained by replacing the L_2 norm with L_1 norm

$$\min_{\boldsymbol{w}} \sum_{i=1}^{m} (y_i - \boldsymbol{w}^\top \boldsymbol{x}_i)^2 + \lambda \|\boldsymbol{w}\|_1$$

An embedding method that is more likely to lead to sparse solutions.

Using L_1 norm is more likely to lead to a sparse solution



Suppose $\boldsymbol{x}$ has only two features, then the solution $\boldsymbol{w}$ has two components w_1 and w_2.
The solutions need to make a **trade-off** between the squared error term and the regularization term.
In other words, the solutions lie on the **intersections** between the contours of the squared error term and the regularization term.

From the figure, when using L_1 norm, the intersections often lie on the axes, that is, either w_1 or w_2 is 0.

A contour is a line composed of points with the same value (error).

Solve the L_1 norm (1)

- The optimization objective with the regularization term is:

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) + \lambda \|\boldsymbol{x}\|_1$$

- To solve it, let us use the method proposed in:

Proximal Gradient Descend (PGD) [Boyd and Vandenberghe, 2004]

- $f(\boldsymbol{x})$, in the univariate case, can be approximated nearby the value x=*a*, better than with a linear extrapolation, by the second-order Taylor expansion:

$$P_2(x) = f(a) + f'(a)(x-a) + \frac{1}{2} f''(a)(x-a)^2$$

Proximal Gradient Descend (PGD)

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) + \lambda \|\boldsymbol{x}\|_1$$

- $f(\boldsymbol{x})$ (in vectorial form) can be approximated by the second-order Taylor expansion:

$$f(\boldsymbol{x}) = f(\boldsymbol{x}_k) + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x} - x_k \rangle + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_k)^\top \frac{\delta^2 f(\boldsymbol{x}_k)}{\delta \boldsymbol{x}_k^2}(\boldsymbol{x} - \boldsymbol{x}_k)$$

- Suppose $f(\boldsymbol{x})$ is differentiable and $\nabla f(\boldsymbol{x})$ satisfies the L-Lipschitz condition, that is: there exists a constant $L > 0$ such that

$$\|\nabla f(\boldsymbol{x}') - \nabla f(\boldsymbol{x})\|_2 \leq L \|\boldsymbol{x}' - \boldsymbol{x}\|_2$$

Using the L-Lipschitz condition (1)

- An **L-Lipschitz condition** is a property of a function $f(x)$ that limits how fast it can change.
- The condition states that for any two points x and x', the absolute difference in the function's output is no more than a constant L times the absolute difference in its inputs, meaning the function's rate of change is bounded:

$$|f(x) - f(x')| \leq L \cdot |x - x'|$$

Using the L-Lipschitz condition (2)

- An **L-Lipschitz condition** is a property of a function $f(x)$ that limits how fast it can change.
- The condition states that for any two points x ans x', the absolute difference in the function's output is no more than a constant L times the absolute difference in its inputs, meaning the function's rate of change is bounded:

$$|f(x) - f(x')| \le L \cdot |x - x'|$$

- Now, suppose $\nabla f(\boldsymbol{x})$ satisfies the L-Lipschitz condition; there exists a constant $L > 0$ such that:

$$\|\nabla f(\boldsymbol{x}') - \nabla f(\boldsymbol{x})\|_2 \le L\|\boldsymbol{x}' - \boldsymbol{x}\|_2$$

Solve the L_1 norm (2)

- Replace the L-Lipschitz condition:

$$\|\nabla f(\boldsymbol{x}') - \nabla f(\boldsymbol{x})\|_2 \leq L\|\boldsymbol{x}' - \boldsymbol{x}\|_2$$

into the Taylor expansion:

$$f(\boldsymbol{x}) = f(\boldsymbol{x}_k) + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x} - x_k \rangle + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_k)^\top \frac{\delta^2 f(\boldsymbol{x}_k)}{\delta \boldsymbol{x}_k^2}(\boldsymbol{x} - \boldsymbol{x}_k)$$

- We obtain:

$$f(\boldsymbol{x}) \cong f(\boldsymbol{x}_k) + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x} - \boldsymbol{x}_k \rangle + \frac{L}{2}\|\boldsymbol{x} - \boldsymbol{x}_k\|^2$$

$$= f(x_k) + \nabla f(x_k) \cdot (x - x_k) + \frac{L}{2}(x - x_k)^2$$

$$= f(x_k) + \underbrace{\frac{L}{2}[\frac{2}{L}\nabla f(x_k) + (x - x_k)]^2}$$

$$= \frac{L}{2}\|\boldsymbol{x} - (\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k))\|^2 + \text{const}$$

Solve the L_1 norm (3)

- Replace the approximation of $f(\boldsymbol{x})$ into the optimization objective:

$$\min_{\boldsymbol{x}} \sum_{i=1}^{m} \frac{L}{2} \left\| \boldsymbol{x} - \left(\boldsymbol{x}_k - \frac{1}{L} \nabla f(\boldsymbol{x}_k)\right) \right\|^2 + \lambda \|\boldsymbol{x}\|_1$$

- The minimum of the quadratic term is at:

$$\boldsymbol{x}_{\boldsymbol{k+1}} = \boldsymbol{x}_k - \frac{1}{L} \nabla f(\boldsymbol{x}_k)$$

- We solve by gradient descent of $\nabla f(\boldsymbol{x}_k)$. Each iteration ($i$) aims to find an optimal solution nearby $\boldsymbol{x}_k$

$$\boldsymbol{x}_{k+1} = \min_{\boldsymbol{x}} \sum_{i=1}^{m} \frac{L}{2} \Big\| \boldsymbol{x} - \underbrace{\left(\boldsymbol{x}_k - \frac{1}{L} \nabla f(\boldsymbol{x}_k)\right)}_{\mathbf{z}} \Big\|^2 + \lambda \|\boldsymbol{x}\|_1$$

- We write $\boldsymbol{z}^i = \boldsymbol{x}_k - \frac{1}{L} \nabla f(\boldsymbol{x}_k)$ the solution found at iteration *i*. There is a closed-form solution:

$$x_{k+1}^i = \begin{cases} z^i - \lambda/L, & \lambda/L < z^i; \\ 0, & |z^i| \leqslant \lambda/L; \\ z^i + \lambda/L, & z^i < -\lambda/L, \end{cases}$$

Sparse representation

- Consider the data set as a matrix, in which each row corresponds to a sample, and each column corresponds to a feature
- There are many zero elements in the matrix D, but the zero elements do not lie in the whole rows or columns
- Advantages of sparse representation:
  - Make many text classification problems **linearly separable**
  - **Less storage** overheads

Is it possible to convert a dense data set into an appropriately sparse representation to take its advantages?

Dictionary learning

The process of constructing an appropriate *dictionary* to convert samples into a sparse representation is called *dictionary learning*

- Given a data set $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\},\ \boldsymbol{x}_i \in \mathbb{R}^{n\times k}$

- Aim to learn:
  - a dictionary matrix $\mathbf{B} \in \mathbb{R}^{d\times k}$ and
  - for each sample $\boldsymbol{x_i}$ a sparse representation $\boldsymbol{\alpha}_i \in \mathbb{R}^k$

  where k is the vocabulary size (usually specified by the user)

- The simplest form of dictionary learning is:

$$\min_{\mathbf{B},\boldsymbol{\alpha}_i} \sum_{i=1}^{m} \|\mathbf{x}_i - \mathbf{B}\boldsymbol{\alpha}_i\|_2^2 + \lambda \sum_{i=1}^{m} \|\boldsymbol{\alpha}_i\|_1$$

Solve the dictionary learning (1)

- Inspired by LASSO, the dictionary learning can be solved by alternating optimization:

1. Having fixed the dictionary $\mathbf{B}$, finds $\boldsymbol{\alpha}_i$:

$$\min_{\boldsymbol{\alpha}_i} \|\boldsymbol{x}_i - \mathbf{B}\boldsymbol{\alpha}_i\|_2^2 + \lambda \|\boldsymbol{\alpha}_i\|_1$$

2. The found $\boldsymbol{\alpha}_i$ are used to optimize the dictionary $\mathbf{B}$:

$$\min_{\mathbf{B}} \|\mathbf{X} - \mathbf{B}\mathbf{A}\|_F^2$$

$||\cdot||_F$ is the Frobenius norm:

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}|a_{ij}|^2} =$$

Solve the dictionary learning by KSVD (1)

- KSVD [Aharon et al., 2006] is a generalization of K-means
- It takes a column-wise updating strategy of matrix ***B*** (updates only one column of B in each iteration) to solve the optimization task:

$$\min_{\mathbf{B}} \|\mathbf{X}-\mathbf{B}\mathbf{A}\|_F^2 = \min_{\boldsymbol{b}_i} \left\|\mathbf{X}-\sum_{j=1}^{k}\boldsymbol{b}_j\boldsymbol{\alpha}^j\right\|_F^2$$

Solve the dictionary learning by KSVD (2)

- The formula can be rewritten as:

$$\underset{\text{goal}\rightarrow\ \boldsymbol{b}_i}{\min} \left\| \left(\mathbf{X} - \underbrace{\sum_{j \neq i} \boldsymbol{b}_j \boldsymbol{\alpha}^j}_{\text{kept fixed}} \right) - \boldsymbol{b}_i \boldsymbol{\alpha}^i \right\|_F^2 = \min_{\boldsymbol{b}_i} \left\| \mathbf{E}_i - \boldsymbol{b}_i \boldsymbol{\alpha}^i \right\|_F^2$$

- Using the singular value decomposition of the fixed first term, it finds the orthogonal vectors corresponding to the largest singular values of $\mathbf{E}_i$

To preserve the sparsity, only the non-zero elements α^i are preserved and E_i only preserves the non-zero product terms of b_i and α^i .

- Iterate several times to generate the optimal solution.

Compressed Sensing

Recover the full information from the partially available information.

- In the telecommunication, as example, can we fully reconstruct the original signal from the converted digital signals, if the digital signal is compressed and suffers from packet loss?

- One solution to such problems is *compressive sensing* [Cándes et al., 2006, Donoho, 2006].

Compressed Sensing

- Suppose there is a discrete signal $\boldsymbol{x}$ with the length m;
  - by sampling at a rate much lower than the sampling rate required by the Nyquist sampling theorem(*), we obtain a sampled signal $\boldsymbol{y}$ with a length n ($n \ll m$):

$$\boldsymbol{y} = \boldsymbol{\Phi x}$$

Measurement matrix

- In general, $\boldsymbol{x}$ cannot be recovered from $\boldsymbol{y}$ since $n \ll m$

- However, if there exists a **linear transformation** Ψ that satisfies $\boldsymbol{x} = \Psi \boldsymbol{s}$ and $\boldsymbol{s}$ is a sparse vector

$$\boldsymbol{y} = \boldsymbol{\Phi \Psi s} = \mathbf{A}\boldsymbol{s}$$

Fourier transform,
cosine transform,
wavelet transform etc.

when A has the *Restricted Isometry Property*, $\boldsymbol{s}$ can be almost fully reconstructed.

(*) The Nyquist sampling theorem states the conditions for a full reconstruction of the signal from samples: the sampling frequency is at least twice the highest frequency of the analog signal

Restricted Isometry Property

Restricted Isometry Property, RIP [Cándes, 2008]:

For a $n \times m$ $(n \ll m)$ matrix $\mathbf{A}$, we say $\mathbf{A}$ satisfies the k-restricted isometry property (k-RIP) if there exists a constant $\delta_k \in (0,1)$ such that

$$(1-\delta_k)\|\boldsymbol{s}\|_2^2 \leqslant \|\mathbf{A}_k\boldsymbol{s}\|_2^2 \leqslant (1+\delta_k)\|\boldsymbol{s}\|_2^2$$

where $\boldsymbol{A_k}$ is a submatrix of A.

The optimization object and the solution to compressed sensing

If $\mathbf{A}$ satisfies k-RIP, $\boldsymbol{s}$ can be almost fully reconstructed from $\boldsymbol{y}$, and subsequently $\boldsymbol{x}$:

$$\min_{\boldsymbol{s}} \quad \|\boldsymbol{s}\|_0$$
$$\text{s.t.} \quad \boldsymbol{y} = \mathbf{A}\boldsymbol{s}$$

the number of non-zero elements in a vector

- Minimize the L_0 norm is a NP-hard problem. Fortunately, under certain conditions, minimizing L_0 norm shares the same solution with minimizing L_1 norm [Cándes et al., 2006] :

$$\min_{\boldsymbol{s}} \quad \|\boldsymbol{s}\|_1$$
$$\text{s.t.} \quad \boldsymbol{y} = \mathbf{A}\boldsymbol{s}$$

- This formula can be converted to a form that is equivalent to LASSO, which can then be solved by proximal gradient descent, that is, using Basis Pursuit De-Noising [Chen et al., 1998]

Matrix completion

The book ratings by some readers

	Gulliver's Travels	Robinson Crusoe	The Story of Mankind	A Study of History	Gitanjali
Emma	5	?	?	3	2
Michael	?	5	3	?	5
John	5	3	?	?	?
Sarah	3	?	5	4	?

Considering the ratings as a **partially available signal**, is it possible to **recover the original signal** via the mechanism of compressed sensing? In general, the preference of readers has a **sparse representation**. Thus, the answer is "**YES**".

Matrix completion can be used to solve this problem.

The optimization object and the solution to matrix completion

- The optimization object of matrix completion is:

$$\min_{\mathbf{X}} \ \text{rank}(\mathbf{X})$$
$$\text{s.t.} \ (\mathbf{X})_{ij} = (\mathbf{A})_{ij}, \ (i, j) \in \Omega$$

- $\mathbf{X}$: the signal to be recovered
- $\text{rank}(\mathbf{X})$: the rank of the matrix
- $\mathbf{A}$: the observed signal
- Ω: the set of subscript index of the non-missing elements in $\mathbf{A}$

The constraint indicates that the recovered matrix should not change the non-missing elements.

- NP-hard. Use the *nuclear norm* to approximate it:

$$\min_{\mathbf{X}} \ \|\mathbf{X}\|_*$$
$$\text{s.t.} \ (\mathbf{X})_{ij} = (\mathbf{A})_{ij}, \ (i, j) \in \Omega.$$

True ratings

Observed ratings

$||X||_*$ indicates the *nuclear norm*, which is the sum of singular values of the matrix

- It is a convex optimization problem that can be solved with Semi-Definite Programming (SDP).

Under certain conditions, $\mathbf{A}$ can be perfectly reconstructed with $O(mr\log^2 m)$ observed elements. [Recht, 2011]

rank(A)

number of true ratings

Conclusion

- Feature selection: subset search & subset evaluation
  - Filter methods
  - Wrapper methods
  - Embedding methods: regularization with L_1 norm
- Sparse representation
  - Dictionary learning
  - Compressed sensing

Thanks!